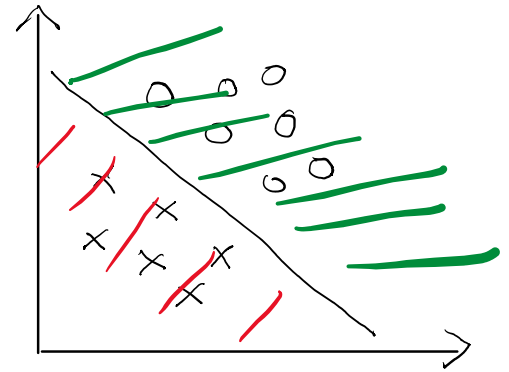
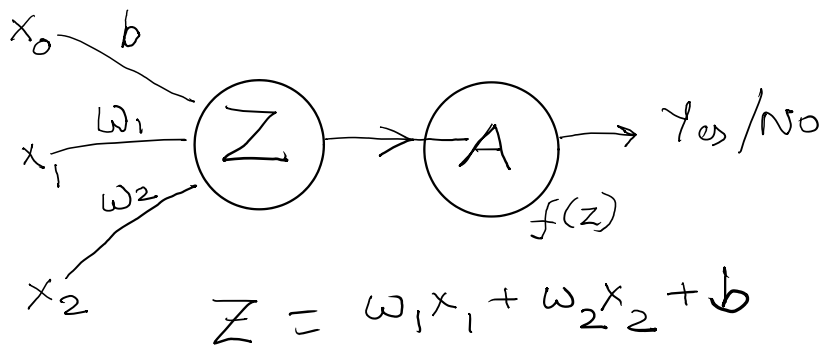


Perceptron

11 April 2026 04:46 PM

We want to predict if Loan will be approved or not based on 2 features:

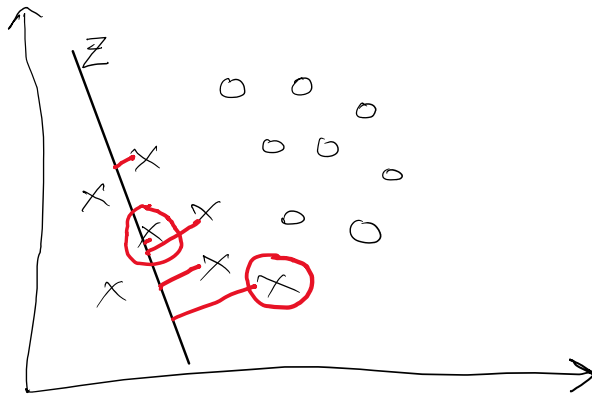
- Credit Score (x_1)
- Salary (x_2)



$$y = f(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

Loss Function

11 April 2026 09:59 PM



Loss (ex 1) $\rightarrow$ 5 misclassified data points

$$L(w_1, w_2, b) = \frac{1}{n} \sum_{i=1}^n \max(0, [-y_i z_i])$$

Only misclassified data points contribute to the loss function

$$Z = w_1 x_1 + w_2 x_2 + b$$

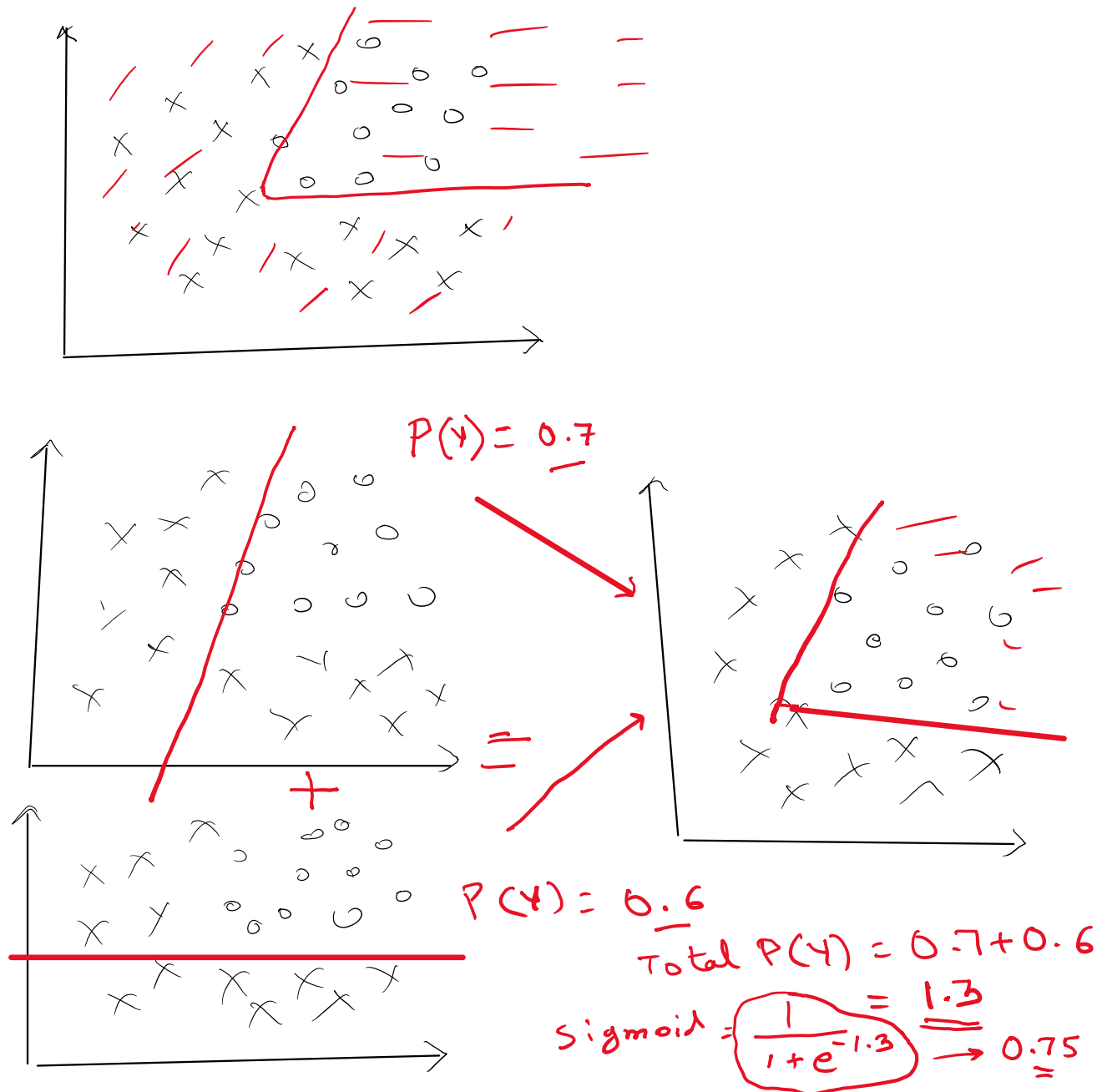
$$W_{\text{new}} = W_{\text{old}} + (\text{LR}) * (\text{Loss})$$

LR $\rightarrow$ Learning Rate (ex. 0.01, 0.1)

Perceptron Problem

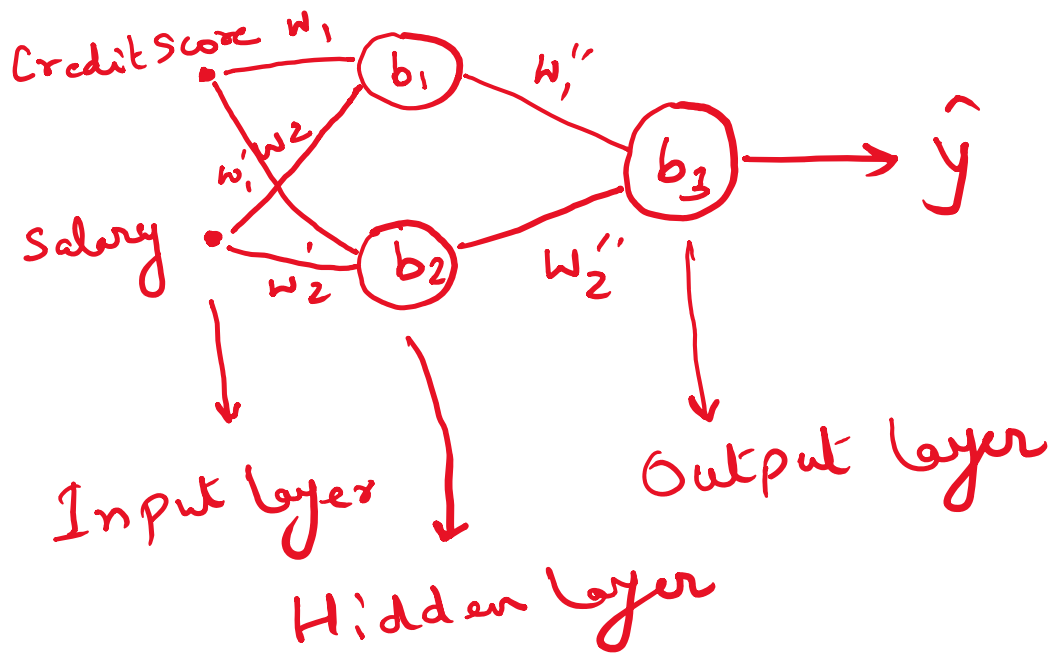
11 April 2026 10:31 PM

Perceptron can work only where the data points can be separated with a linear model



Multilayer Perceptron

11 April 2026 10:44 PM



Forward Propagation

12 April 2026 08:49 PM

Features:

Credit Score (x1)

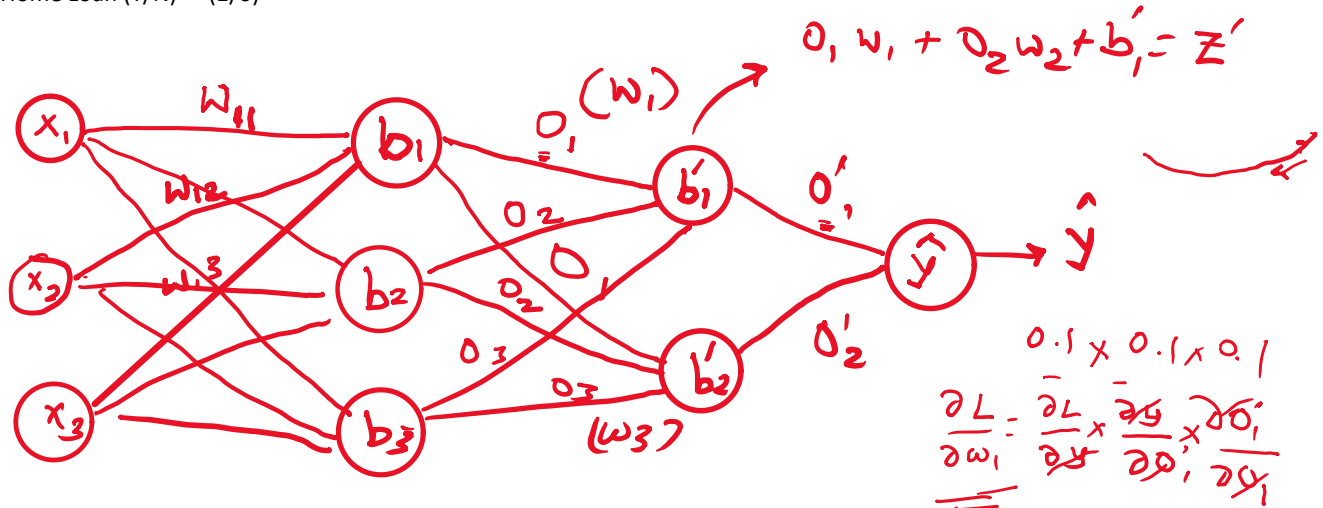
Salary (x2)

Net Worth (x3)

Label: Home Loan (Y/N) -> (1/0)

Total → 1000 rows

Credit Score	Salary	N.W	HL
7.8	50K	3M	1
6.1	25K	1M	0



After an epoch is completed

→ Loss is calculated

→ Weights & biases are adjusted

→ Next epoch is run.

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y} \times \frac{\partial y}{\partial b'_1} \times \frac{\partial b'_1}{\partial w_1}$$

$$= 0.1 \times 0.1 \times 0.1$$

$$= 0.0001$$

$$\rightarrow 0$$

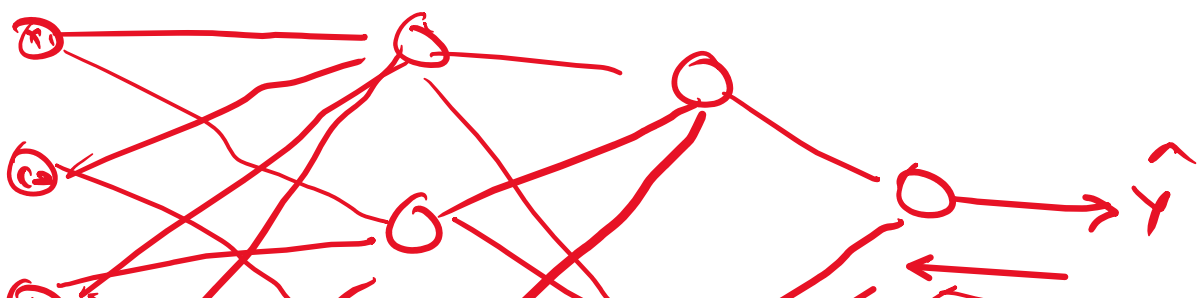
How Many trainable parameters are there:

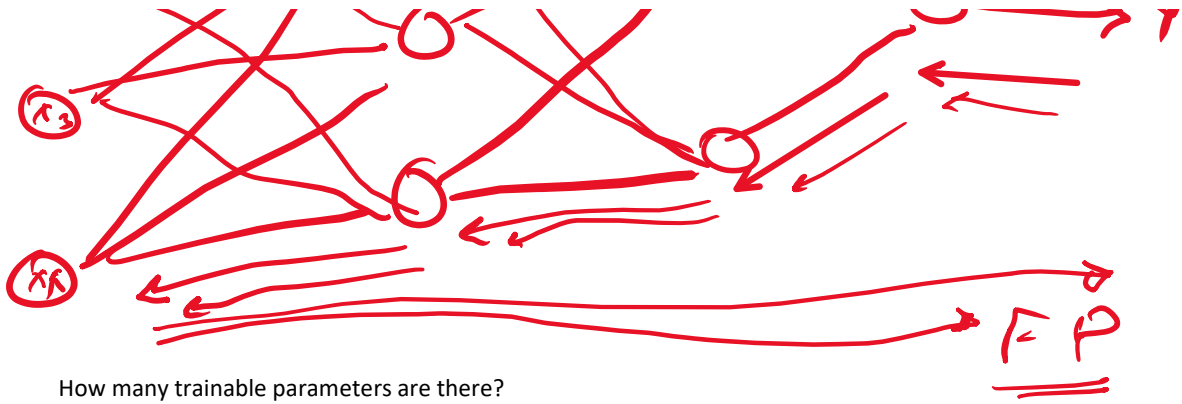
1st Layer -> $3 \times 3 + 3 = 12$

2nd Layer -> $3 \times 2 + 2 = 8$

3rd Layer -> $2 + 1 = 3$

Total = 23





1st Layer = $4 * 3 + 3 = 15$

2nd Layer = $3 * 2 + 2 = 8$

3rd Layer = $2 + 1 = 3$

Total = 26

Forward Propagation -> making predictions

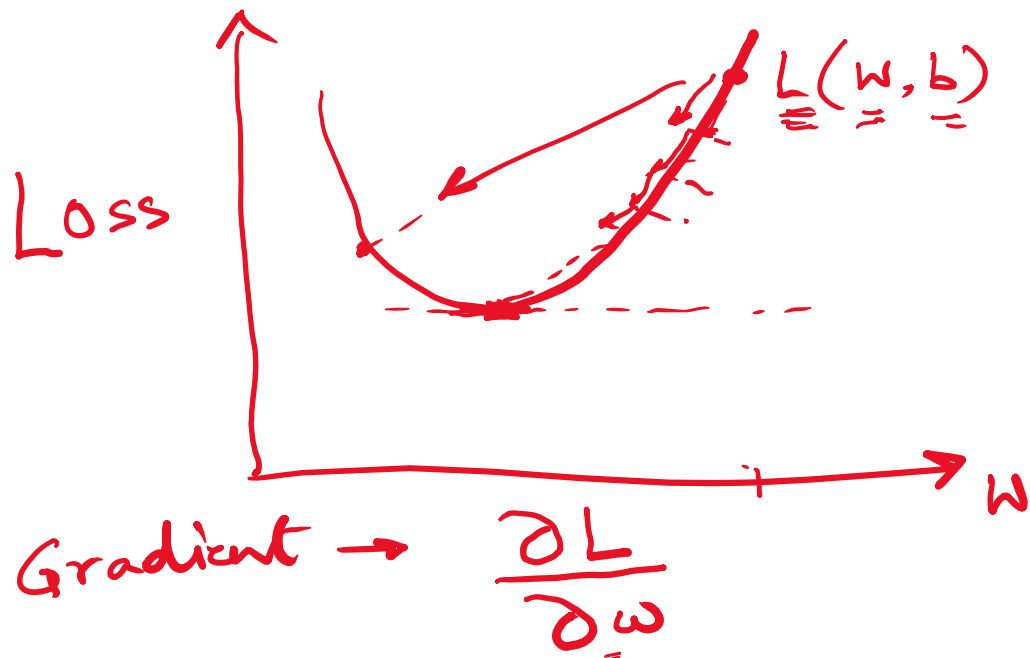
Backpropagation -> Learning for the error, and taking corrective actions.

Backpropagation

12 April 2026 09:39 PM

Gradient Descent -> Engine behind the backpropagation

Loss Function -> $L(w_1, w_2, \dots, b_1, b_2, \dots)$



$w_0 \rightarrow$ initial weight

$$w_1 = w_0 - \eta \times \frac{\partial L}{\partial w}$$

$\hookrightarrow$ Learning Rate

$b_0 \rightarrow$ initial bias

$$b_1 \rightarrow b_0 - \eta \times \frac{\partial L}{\partial b}$$

Example

loss function: $L = (w - 5)^2$

Loss function: $L = (w - 5)^2$

Loss is minimum at $w = 5$

Step 1: Take a random value of w

$$\underline{\underline{w = 0}}$$

Step 2: Calculate Gradient

$$\frac{\partial L}{\partial w} = \frac{\partial [w^2 + \underline{25} - 10\underline{w}]}{\partial w}$$

$$= 2w + 0 - 10$$

$$= 2w - 10$$

$$= \underline{\underline{2(w - 5)}}$$

$$= -10$$

$$w_{\text{new}} = w_{\text{old}(0)} - \eta \times \underline{\underline{\frac{\partial L}{\partial w}}}$$

$$= 0 - 0.1 \times (-10)$$

$$= 1$$

$$w_{\text{old}(1)} = 1$$

$$\frac{\partial L}{\partial w} \Rightarrow -8$$

$$w_{\text{new}(1)} = w_{\text{old}(1)} - \eta \times \frac{\partial L}{\partial w}$$

$$\begin{aligned}
 w_{\text{new}}(1) &= w_{\text{old}}(1) - \frac{\partial L}{\partial w} \\
 &= 1 - 0.1 \times (-8) \\
 &= 1.8
 \end{aligned}$$

Slowly it moves towards 5

How does Gradient Descent works with Backpropagation?

For a Linear Model:

$$Y = mx + b$$

$$Y = 3x + 1$$

$$\text{Input } (x) = 2$$

$$\text{Output} = 3 \times 2 + 1 = 7$$

Suppose:

Actual Value = 10

Predicted Value = 7

Loss (Mean Squared Error)

$$L = [y(\text{actual}) - y(\text{pred})]^2$$

$$\text{Loss} = (10 - 7)^2 = 9$$

What Back propagation does?

It computes:

Gradient of loss w.r.t weight

Gradient of loss w.r.t bias

$$\frac{\partial L}{\partial w} \rightarrow \frac{\partial L}{\partial y} \times \frac{\partial y}{\partial w}$$

$$\begin{aligned}
 &= \frac{(y - y_p)^2}{(y_p - y)^2} \\
 &= \frac{(y - y_p)^2}{(y - y_p)^2} \rightarrow 2(y - y_p)
 \end{aligned}$$

$$y = w \cdot x + 1$$

$$\begin{aligned}
 \frac{\partial L}{\partial w} &\rightarrow \left(\frac{\partial L}{\partial y} \right) \times \frac{\partial y}{\partial w} \\
 &= 2 \times (y - y_p) \times x \\
 &= 2(10 - 7) \times 2 \\
 &= 2 \times 3 \times 2 \\
 &= \underline{12}
 \end{aligned}$$

$$\begin{aligned}
 \frac{\partial L}{\partial b} &\rightarrow \left(\frac{\partial L}{\partial y} \right) \times \frac{\partial y}{\partial b} \\
 &= (2 \times 3) \times 1 \\
 &= 6
 \end{aligned}$$

$$y = wx + b$$

Diagram showing the equation $y = wx + b$ with arrows indicating the partial derivatives: an arrow from w to $\frac{\partial y}{\partial w}$ and an arrow from b to $\frac{\partial y}{\partial b}$.

$$w_{\text{new}} = w_{\text{old}} + \eta \times \frac{\partial L}{\partial w}$$

$$b_{\text{new}} = b_{\text{old}} + \eta \times \frac{\partial L}{\partial b}$$

$$w_{\text{new}} = 3 + 0.1 \times (12)$$

$$= 3 + 1.2$$

$$= \cancel{1.8} 4.2$$

$$b = 1 + 0.1 \times 6$$

$$b = 1 + 0.1x^2$$

$$= 1 + 0.6$$

$$= \cancel{0.4} + 1.6$$

$$y_{\text{Pred}(2)} = \cancel{1.8x} + \cancel{0.4} \quad 4.2x + 1.6$$

$$= \cancel{1.8x2} + \cancel{0.4} = 4.2x2 + 1.6$$

$$= \cancel{3.6} + \cancel{0.4} = 8.4 + 1.6$$

$$= \cancel{4} = \underline{10}$$

Gradient Descent Problems

18 April 2026 08:15 PM

1. Vanishing Gradient:

Gradient $\rightarrow$ Very small

Learning becomes very slow or negligible

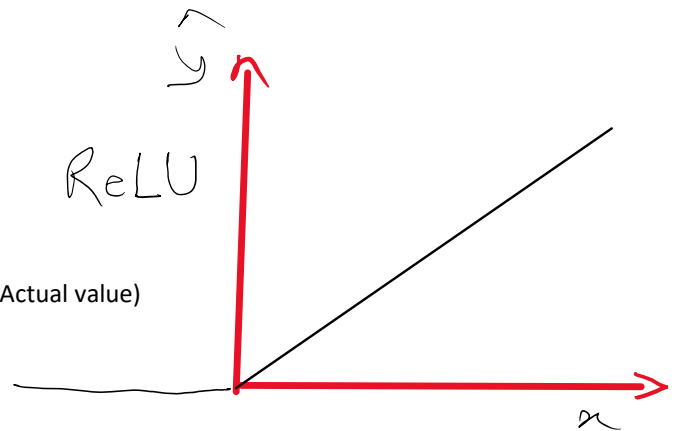
2. Exploding Gradient

Gradient $\rightarrow$ Very large

Training becomes unstable

Solutions:

1. ReLU Activation (when output is < 0 , then 0, when it is > 0 , Actual value)
2. Proper Weights Initialization
3. Batch Normalization
4. Gradient Clipping (solves Exploding Gradient Problems)



Convolution Neural Network (CNN)

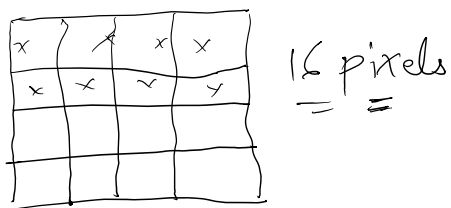
18 April 2026 08:51 PM

Convolution Neural Network is used to work on GRID like data. It works very well on images.

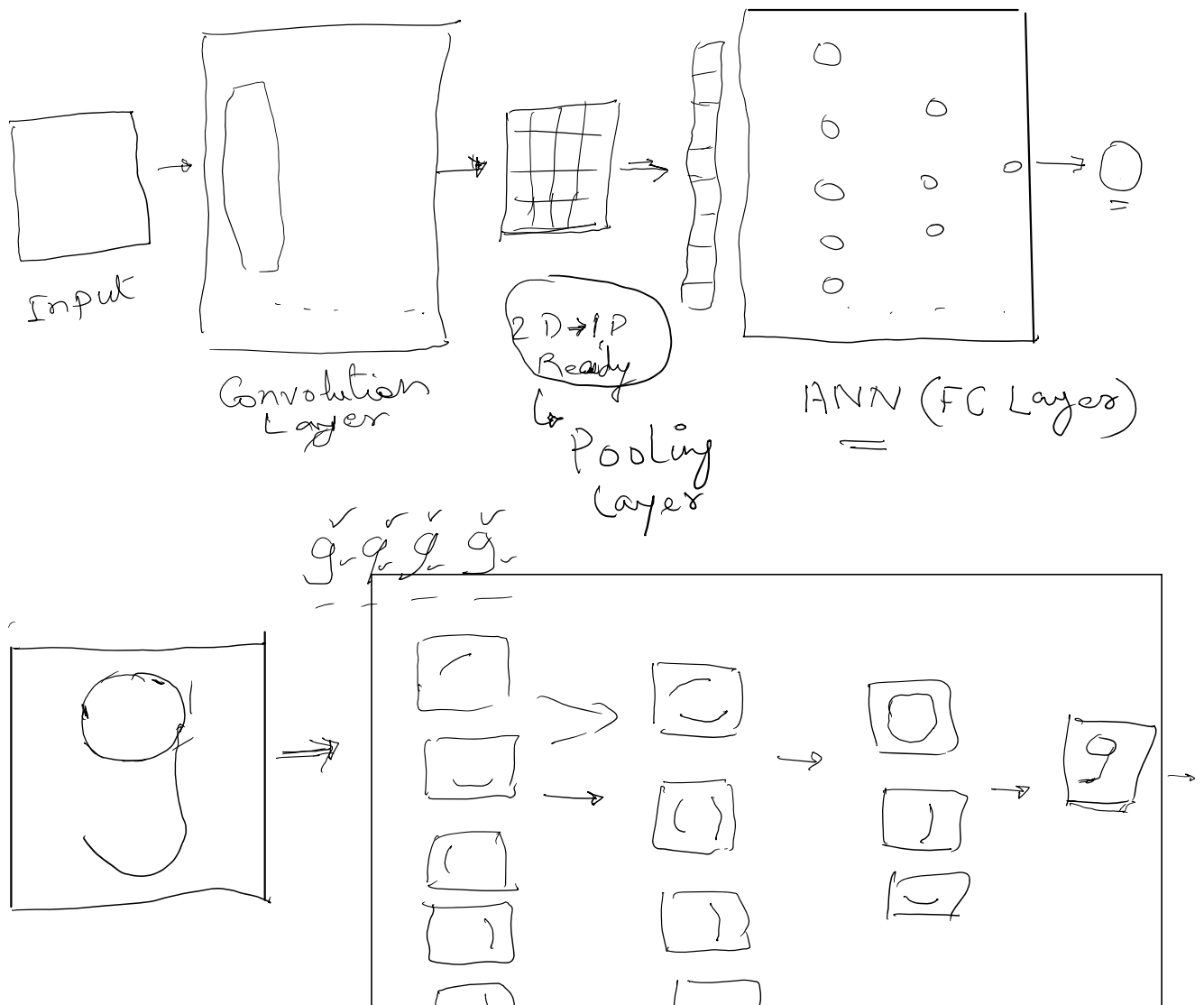
Examples:

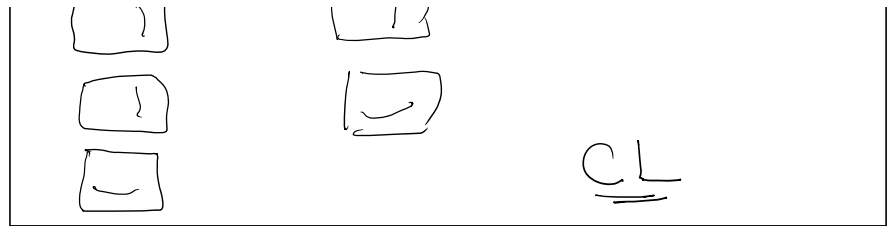
1. Face Recognition
2. Image Recognition (if an image is of a DOG or CAT or AIRPLANE)
3. Object Detection - Self Driven Vehicles
4. Healthcare / medical Imaging - CT scans / Xrays / MRIs analysis
5. Video Analysis

IMAGE -> PIXELS



CNN Architecture:



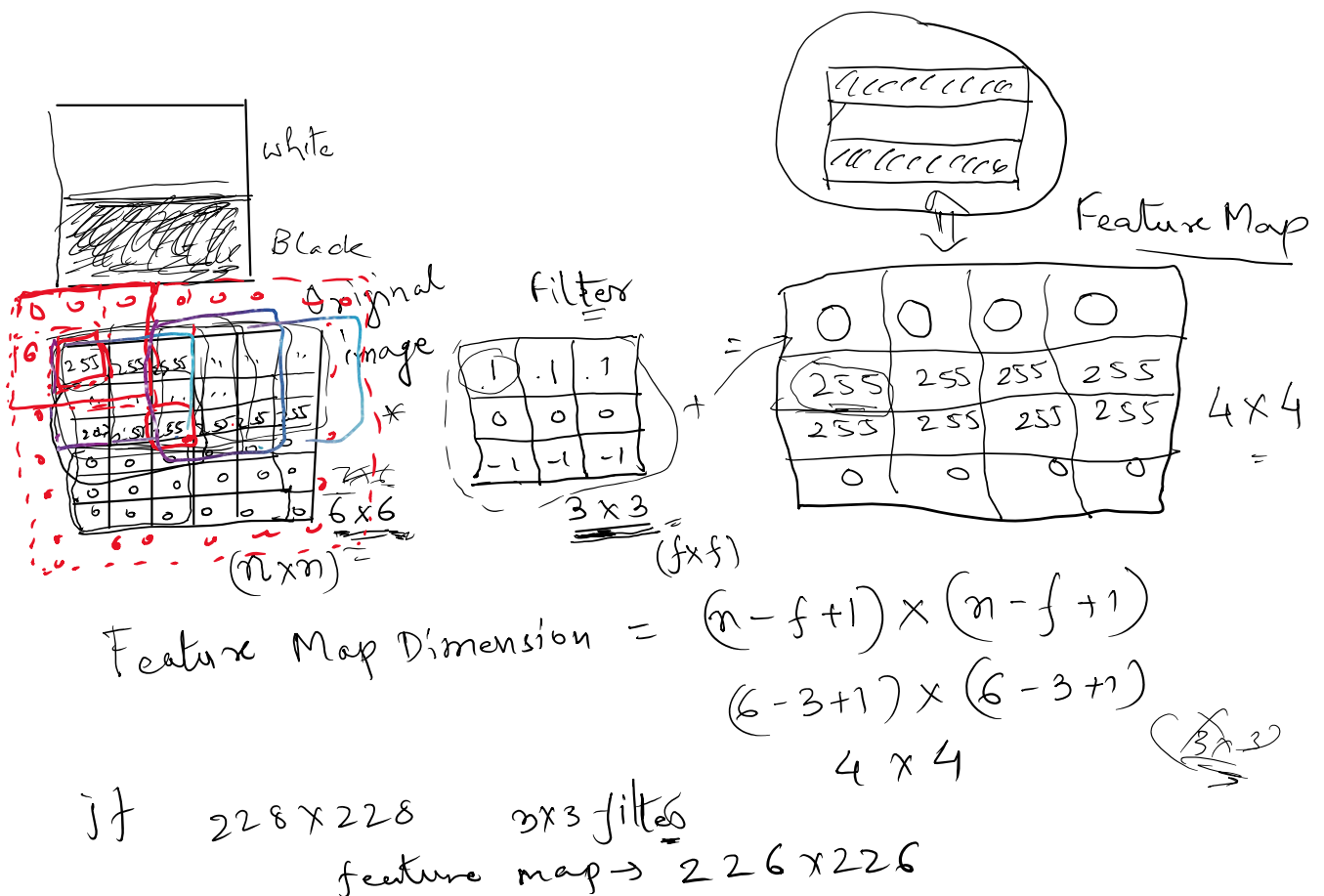


How images are perceived?

- Every image is segregated by pixels with some values.
- Pixel range : 0 -> 255 (0 = Black color; 255 = 255 White color)
- For colored images : pixel values are distributed in 3 channels (RGB Channels)

FILTER (Kernel):

A filter is a small matrix that slides over the image to detect features like edges, shapes, textures etc)



Problems with CNN:

1. Pixels in the feature map reduces with each layer of network.
2. Some pixels are considered more than the other pixels (usually pixels at the center)

Solution:

PADDING

Padding is adding extra pixels around the image border.

In KERAs (deep learning library in Python), there is a parameter called PADDING:

- 1) VALID -> means don't apply padding.
- 2) SAME -> means apply padding.

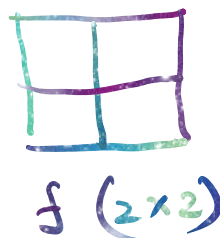
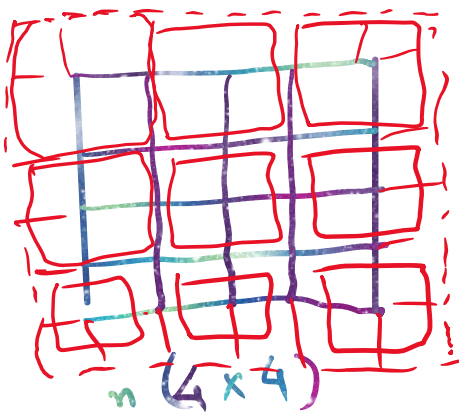
STRIDE - Steps to move in the right or bottom direction

STRIDE -> 1,1 -> 1 step to the right and 1 step to the bottom

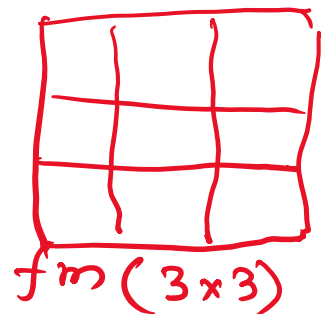
STRIDE -> 1,2 -> 1 step to the right but 2 steps to the bottom.

Usually we go with 1X1 stride or 2X2 stride.

Feature MAP Dimension Formula = $\lceil (n - f + 2p) / s \rceil + 1$



Before Padding & Stride(1)



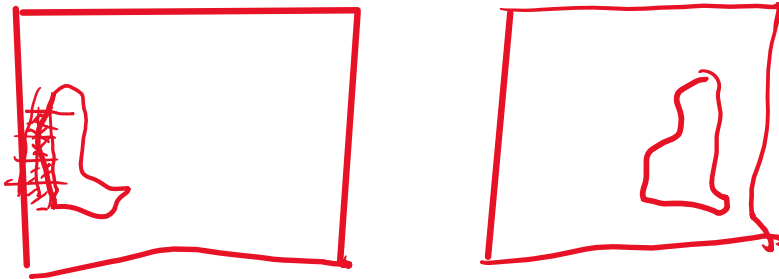
$$\begin{aligned} n - f + 1 \\ &= 4 - 2 + 1 \\ &= 3 \end{aligned}$$

$$n (6 \times 6)$$

$$\begin{aligned} f_m &= n - f + 2p + 1 \\ &= 4 - 2 + 2 + 1 \\ &= 5 \end{aligned}$$

$$\text{stride} \rightarrow 2, 2$$

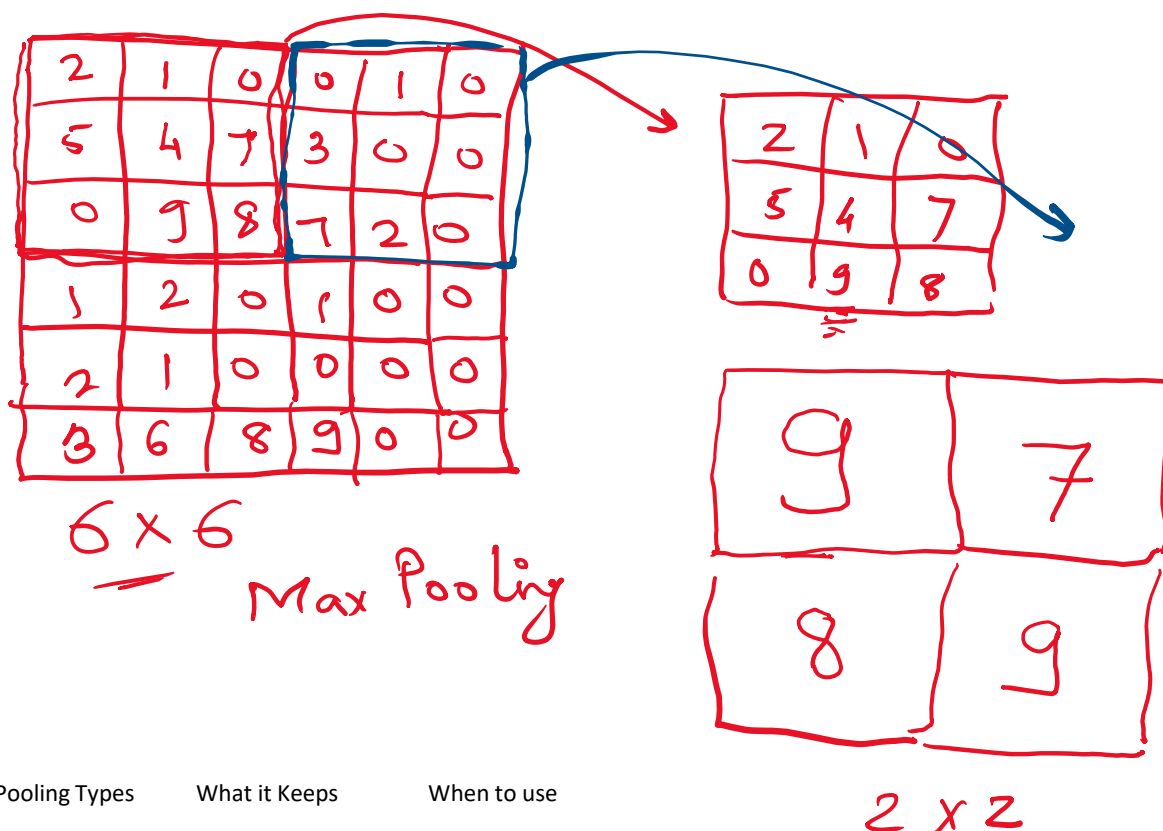
$$\begin{aligned} f_m &= \left\lceil \frac{n - f + 2p}{s} \right\rceil + 1 \\ &= \left\lceil \frac{4 - 2 + 2}{2} \right\rceil + 1 = 2 + 1 = 3 \\ &\quad (3 \times 3) \end{aligned}$$



POOLING:

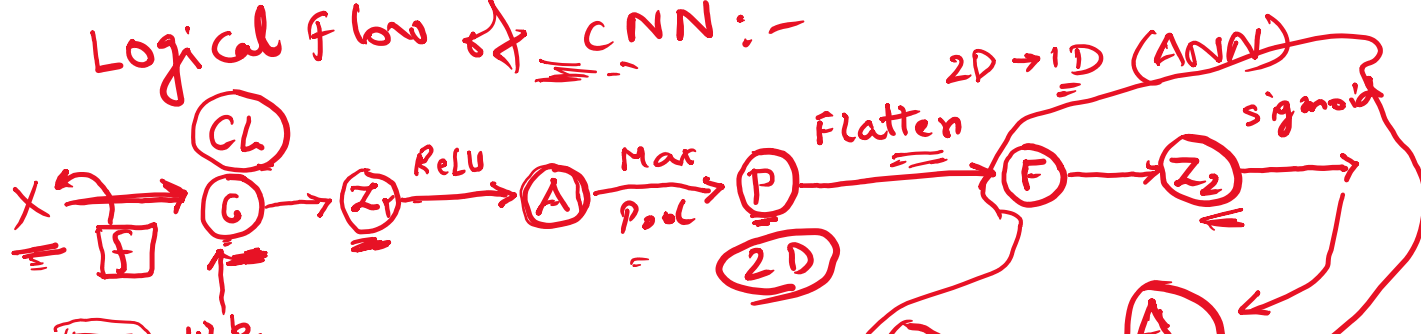
Pooling is a technique used in CNN to:

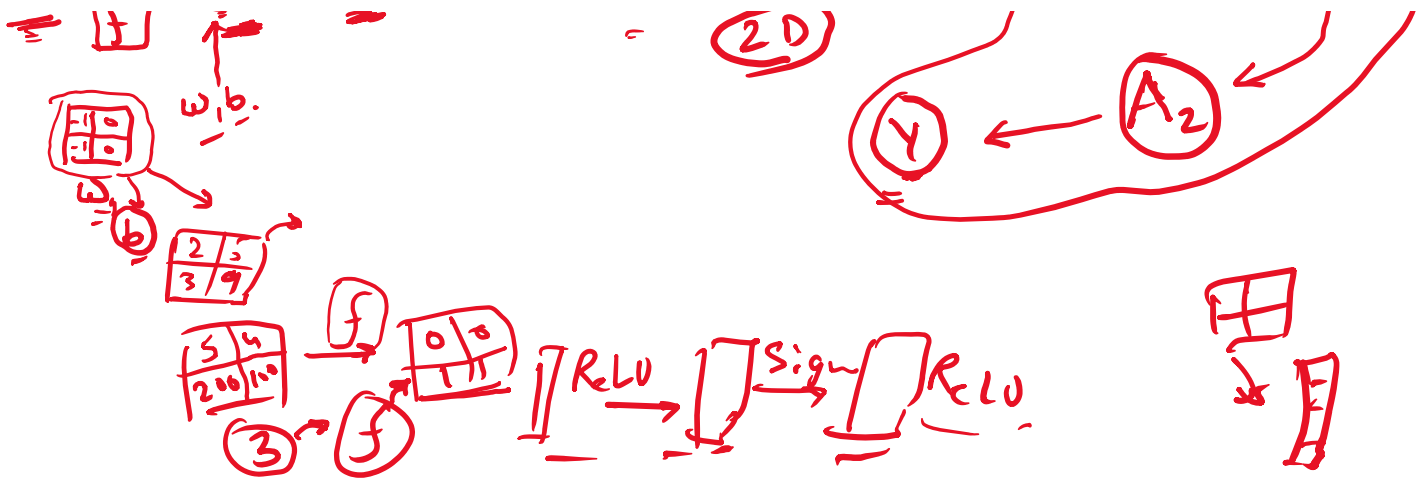
- Reducing the size of the feature map
- Identify the key features with overall context of the image



Pooling Types	What it Keeps	When to use
MaxPooling	Strongest Feature	Edge/Feature detection
Avg Pooling	Overall Pattern	Overall understanding / generalizing
MinPooling	Weakest Feature	Rare / special cases

Logical flow of CNN :-





Types of CNN Architectures:

1. LeNET
2. AlexNET
3. GoogleNET
4. VGGNET
5. ResNET



Optimization happens with Backpropagation

